

10

Inference Pipeline Deployment

Deploying the inference pipeline for the **large language model (LLM)** Twin application is a critical stage in the **machine learning (ML)** application life cycle. It's where the most value is added to your business, making your models accessible to your end users. However, successfully deploying AI models can be challenging, as the models require expensive computing power and access to up-to-date features to run the inference. To overcome these constraints, it's crucial to carefully design your deployment strategy. This ensures that it meets the application's requirements, such as latency, throughput, and costs. As we work with LLMs, we must consider the inference optimization techniques presented in *Chapter 8*, such as model quantization. Also, to automate the deployment processes, we must leverage MLOps best practices, such as model registries that version and share our models across our infrastructure.

To understand how to design the deployment architecture of the LLM Twin, we will first look at three deployment types we can choose from: online real-time inference, asynchronous inference, and offline batch transform. Also, to better understand which option to choose for our LLM Twin use case, we will quickly walk you through a set of critical criteria we must consider before making an architectural decision, such as latency, throughput, data, and infrastructure. Also, we'll weigh the pros and cons of monolithic and microservices architecture in model serving, a decision that can significantly influence the scalability and maintainability of your service. Once we've grasped the various design choices available, we'll focus on understanding the deployment strategy for the LLM Twin's inference pipeline. Subsequently, we will walk you through an end-to-end tutorial on deploying the LLM Twin service, including deploying our custom fine-tuned LLM to AWS SageMaker endpoints and implementing a FastAPI server as the central entry point for our users. We will then wrap up this chapter with a short discussion on autoscaling strategies and how to use them on SageMaker.

Hence, in this chapter, we will cover the following topics:

- Criteria for choosing deployment types
- Understanding inference deployment types
- Monolithic versus microservices architecture in model serving
- Exploring the LLM Twin's inference pipeline deployment strategy
- Deploying the LLM Twin service
- Autoscaling capabilities to handle spikes in usage

Criteria for choosing deployment types

When it comes to deploying ML models, the first step is to understand the four requirements present in every ML application: throughput, latency, data, and infrastructure.

Understanding them and their interaction is essential. When designing the deployment architecture for your models, there is always a trade-off

between the four that will directly impact the user's experience. For example, should your model deployment be optimized for low latency or high throughput?

Throughput and latency

Throughput refers to the number of inference requests a system can process in a given period. It is typically measured in **requests per second (RPS)**. Throughput is crucial when deploying ML models when you expect to process many requests. It ensures the system can handle many requests efficiently without becoming a bottleneck.

High throughput often requires scalable and robust infrastructure, such as machines or clusters with multiple high-end GPUs. **Latency** is the time it takes for a system to process a single inference request from when it is received until the result is returned. Latency is critical in real-time applications where quick response times are essential, such as in live user interactions, fraud detection, or any system requiring immediate feedback. For example, the average latency of OpenAI's API is the average response time from when a user sends a request, and the service provides a result that is accessible within your application.

The latency is the sum of the network I/O, serialization and deserialization, and the LLM's inference time. Meanwhile, the throughput is the average number of requests the API processes and serves a second.

Low-latency systems require optimized and often more costly infrastructure, such as faster processors, lower network latency, and possibly edge computing to reduce the distance data needs to travel.

A lower latency translates to higher throughput when the service processes multiple queries in parallel successfully. For example, if the service takes 100 ms to process requests, this translates to a throughput of 10 requests per second. If the latency reaches 10 ms per request, the throughput rises to 100 requests per second.

However, to complicate things, most ML applications adopt a batching strategy to simultaneously pass multiple data samples to the model. In this case, a lower latency can translate into lower throughput; in other words, a higher latency maps to a higher throughput. For example, if you process 20 batched requests in 100 ms, the latency is 100 ms, while the throughput is 200 requests per second. If you process 60 requests in 200 ms, the latency is 200 ms, while the throughput rises to 300 requests per second. Thus, even when batching requests at serving time, it's essential to consider the minimum latency accepted for a good user experience.

Data

As we know, data is everywhere in an ML system. But when talking about model serving, we mostly care about the model's input and output. This includes the format, volume, and complexity of the processed data. Data is the foundation of the inference process. The characteristics of the data, such as its size and type, determine how the system needs to be configured and optimized for efficient processing.

The type and size of the data directly impact latency and throughput, as more complex or extensive data can take longer to process. For example,

designing a model that takes input structured data and outputs a probability differs entirely from an LLM that takes input text (or even images) and outputs an array of characters.

Infrastructure

Infrastructure refers to the underlying hardware, software, networking, and system architecture that supports the deployment and operation of the ML models. The infrastructure provides the necessary resources for deploying, scaling, and maintaining ML models. It includes computing resources, memory, storage, networking components, and the software stack:

- For **high throughput**, the systems require scalable infrastructure to manage large data volumes and high request rates, possibly through parallel processing, distributed systems, and high-end GPUs.
- Infrastructure must be optimized to reduce processing time to achieve **low latency**, such as using faster CPUs, GPUs, or specialized hardware. While optimizing your system for low latency while batching your requests, you often have to sacrifice high throughput in favor of lower latency, which can result in your hardware not being utilized at total capacity. As you process fewer requests per second, it results in idle computing, which increases the overall cost of processing a request. Thus, picking the suitable machine for your requirements is critical in optimizing costs.

It is crucial to design infrastructure to meet specific data requirements. This includes selecting storage solutions to handle large datasets and implementing fast retrieval mechanisms to ensure efficient data access. For example, we mostly care about optimizing throughput for offline training, while for online inference, we generally care about latency.

With this in mind, before picking a specific deployment type, you should ask yourself questions such as:

- What are the throughput requirements? You should make this decision based on the throughput's required minimum, average, and maximum statistics.
- How many requests the system must handle simultaneously? (1, 10, 1,000, 1 million, etc.)
- What are the latency requirements? (1 millisecond, 10 milliseconds, 1 second, etc.)
- How should the system scale? For example, we should look at the CPU workload, number of requests, queue size, data size, or a combination of them.
- What are the cost requirements? With what data do we work with? For example, do we work with images, text, or tabular data?
- What is the size of the data we work with? (100 MB, 1 GB, 10 GB)

Deeply thinking about these questions directly impacts the user experience of your application, which ultimately makes the difference between a successful product and not. Even if you ship a mind-blowing model, if the user needs to wait too long for a response or it often crashes, the user will switch your production to something less accurate that works reliably. For example, Google found in a 2016 study that 53% of visits are abandoned if a mobile site takes longer than three seconds to load:

<https://www.thinkwithgoogle.com/consumer-insights/consumer-trends/mobile-site-load-time-statistics/>

Let's move on to the three deployment architectures we can leverage to serve our models.

Understanding inference deployment types

As illustrated in *Figure 10.1*, you can choose from three fundamental deployment types when serving models:

- Online real-time inference
- Asynchronous inference
- Offline batch transform

When selecting one design over the other, there is a trade-off between latency, throughput, and costs. You must consider how the data is accessed and the infrastructure you are working with. Another criterion you have to consider is how the user will interact with the model. For example, will the user use it directly, like a chatbot, or will it be hidden within your system, like a classifier that checks if an input (or output) is safe?

You have to consider the freshness of the predictions as well. For example, serving your model in offline batch mode might be easier to implement if, in your use case, it is OK to consume delayed predictions. Otherwise, you have to serve your model in real-time, which is more infrastructure-demanding. Also, you have to consider your application's traffic. Ask yourself questions such as, "Will the application be constantly used, or will there be spikes in traffic and then flatten out?"

With that in mind, let's explore the three major ML deployment types.

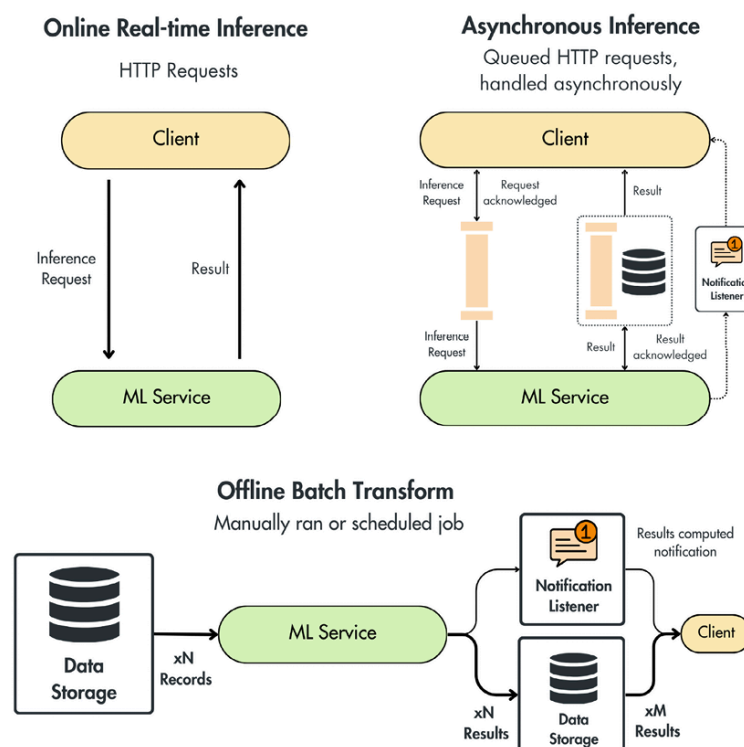


Figure 10.1: The three fundamental architectures of inference deployment types

Online real-time inference

In real-time inference, we have a simple architecture based on a server that can be accessed through HTTP requests. The most popular options are to implement a REST API or gRPC server. The REST API is more accessible but slower, using JSON to pass data between the client and server.

This approach is usually taken when serving models outside your internal network to the broader public. For example, OpenAI's API implements a REST API protocol.

On the other hand, implementing a gRPC makes your ML server faster, though it may reduce its flexibility and general applicability. You have to implement `protobuf` schemas in your client application, which are more tedious to work with than JSON structures. The benefit, however, is that `protobuf` objects can be compiled into bites, making the network transfers much faster. Thus, this protocol is often adopted for internal services within the same ML system.

Using the real-time inference approach, the client sends an HTTP request to the ML service, which immediately processes the request and returns the result in the same response. This synchronous interaction means the client waits for the result before moving on.

To make this work efficiently, the infrastructure must support low-latency, highly responsive ML services, often deployed on fast, scalable servers. Load balancing is crucial to evenly distribute incoming traffic evenly, while autoscaling ensures the system can handle varying loads. High availability is also essential to keeping the service operational at all times.

For example, this architecture is often present when interacting with LLMs, as when sending a request to a chatbot or API (powered by LLMs), you expend the predictions right ahead. LLM services, such as ChatGPT or Claude, often use WebSockets to stream each token individually to the end user, making the interaction more responsive. Other famous examples are AI services such as embedding or reranking models used for **retrieval-augmented generation (RAG)** or online recommendation engines in platforms like TikTok.

The simplicity of real-time inference, with its direct client-server interaction, makes it an attractive option for applications that require immediate responses, like chatbots or real-time recommendations. However, this approach can be challenging to scale and may lead to underutilized resources during low-traffic periods.

Asynchronous inference

In asynchronous inference, the client sends a request to the ML service, which acknowledges the request and places it in a queue for processing. Unlike real-time inference, the client doesn't wait for an immediate response. Instead, the ML service processes the request asynchronously. This requires a robust infrastructure that queues the messages to be processed by the ML service later on.

When the results are ready, you can leverage multiple techniques to send them to the client. For example, depending on the size of the result, you can put it either in a different queue or an object storage dedicated to storing the results.

The client can either adopt a polling mechanism that checks on a schedule if there are new results or adopt a push strategy and implement a notification system to inform the client when the results are ready.

Asynchronous inference uses resources more efficiently. It doesn't have to process all the requests simultaneously but can define a maximum number of machines that run in parallel to process the messages. This is possible because the requests are stored in the queue until a machine can process them. Another huge benefit is that it can handle spikes in requests without any timeouts. For example, let's assume that on an e-shop site, we usually have 10 requests per second handled by two machines. Because of a promotion, many people started to visit the site, and the number of requests spiked to 100 requests per second. Instead of scaling the number of **virtual machines (VMs)** by 10, which can add drastic costs, the requests are queued, and the same two VMs can process them in their rhythm without any failures.

Another popular advantage for asynchronous architectures is when the requested job takes significant time to complete. For example, if the job takes over five minutes, you don't want to block the client waiting for a response.

While asynchronous inference offers significant benefits, it does come with trade-offs. It introduces higher latency, making it less suitable for time-sensitive applications. Additionally, it adds complexity to the implementation and infrastructure. Depending on your design choices, this architecture type falls somewhere between online and offline, offering a balance of benefits and trade-offs.

For example, this is a robust design where you don't care too much about the latency of the inference but want to optimize costs heavily. Thus, it is a popular choice for problems such as extracting keywords from documents, summarizing them using LLMs, or running deep-fake models on top of videos. But suppose you carefully design the autoscaling system to process the requests from the queue at decent speeds. In that case, you can leverage this design for other use cases, such as online recommendations for e-commerce. In the end, it sums up how much computing power you are willing to pay to meet the expectations of your application.

Offline batch transform

Batch transform is about processing large volumes of data simultaneously, either on a schedule or triggered manually. In a batch transform architecture, the ML service pulls data from a storage system, processes it in a single operation, and then stores the results in storage. The storage system can be implemented as an object storage like AWS S3 or a data warehouse like GCP BigQuery.

Unlike the asynchronous inference architecture, a batch transform design is optimized for high throughput with permissive latency requirements. When real-time predictions are unnecessary, this approach can significantly reduce costs, as processing data in big batches is the most economi-

cal method. Moreover, the batch transform architecture is the simplest way to serve a model, accelerating development time.

The client pulls the results directly from data storage, decoupling its interaction with the ML service. Taking this approach, the client never has to wait for the ML service to process its input, but at the same time, it doesn't have the flexibility to ask for new results at any time. You can see the data storage, where the results are stored as a large cache, from where the client can take what is required. If you want to make your application more responsive, the client can be notified when the processing is complete and can retrieve the results.

Unfortunately, this approach will always introduce a delay between the time the predictions were computed and consumed. That's why not all applications can leverage this design choice. For example, if we implement a recommender system for a video streaming application, having a delay of one day for the predicted movies and TV shows might work because you don't consume these products at a high frequency. But suppose you make a recommender system for a social media platform. In that case, delaying one day or even one hour is unacceptable, as you constantly want to provide fresh content to the user.

Batch transform shines in scenarios where high throughput is needed, like data analytics or periodic reporting. However, it's unsuitable for real-time applications due to its high latency and requires careful planning and scheduling to manage large datasets effectively. That's why it is an offline serving method.

To conclude, we examined the three most common architectures for serving ML models. We started with online real-time inference, which serves clients when they request a prediction. Then, we looked at the asynchronous inference method, which sits between online and offline. Ultimately, we presented the offline batch transform, which is used to process large amounts of data and store them in data storage, from where the client later consumes them.

Monolithic versus microservices architecture in model serving

In the previous section, we saw three different methods of deploying the ML service. The differences in architecture were mainly based on the interaction between the client and the ML service, such as the communication protocol, the ML service responsiveness, and prediction freshness.

But another aspect to consider is the architecture of the ML service itself, which can be implemented as a monolithic server or as multiple microservices. This will impact how the ML service is implemented, maintained, and scaled. Let's explore the two options.

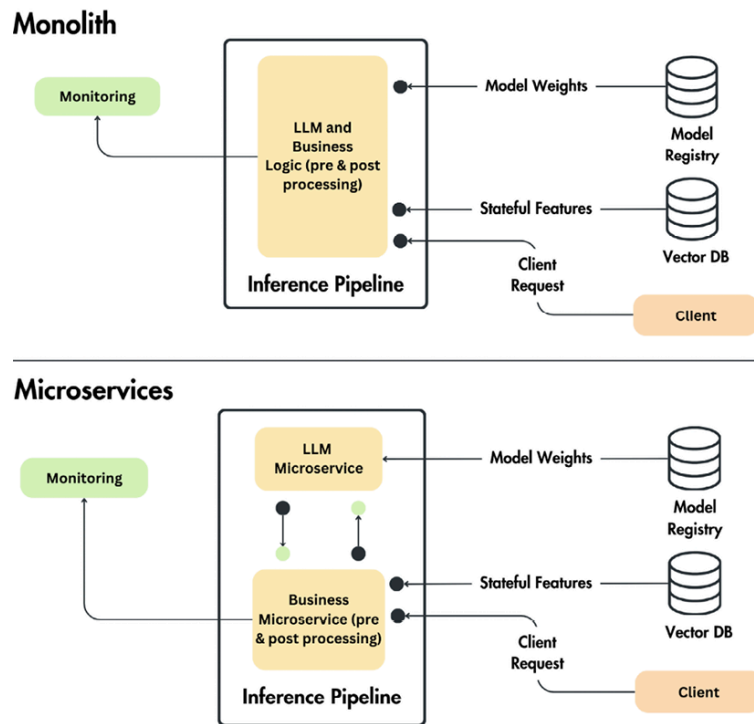


Figure 10.2: Monolithic versus microservices architecture in model serving

Monolithic architecture

The LLM (or any other ML model) and the associated business logic (pre-processing and post-processing steps) are bundled into a single service in a monolithic architecture. This approach is straightforward to implement at the beginning of a project, as everything is placed within one code base. Simplicity makes maintenance easy when working on small to medium projects, as updates and changes can be made within a unified system.

One key challenge of a monolithic architecture is the difficulty of scaling components independently. The LLM typically requires GPU power, while the rest of the business logic is CPU and I/O-bound. As a result, the infrastructure must be optimized for both GPU and CPU. This can lead to inefficient resource use, with the GPU being idle when the business logic is executed and vice versa. Such inefficiency can result in additional costs that could be avoided.

Moreover, this architecture can limit flexibility, as all components must share the same tech stack and runtime environment. For example, you might want to run the LLM using Rust or C++ or compile it with ONNX or TensorRT while keeping the business logic in Python. Having all the code in one system makes this differentiation difficult. Finally, splitting the work across different teams is complex, often leading to bottlenecks and reduced agility.

Microservices architecture

A microservices architecture breaks down the inference pipeline into separate, independent services—typically splitting the LLM service and the business logic into distinct components. These services communicate over a network using protocols such as REST or gRPC.

As illustrated in *Figure 10.3*, the main advantage of this approach is the ability to scale each component independently. For instance, since the LLM service might require more GPU resources than the business logic, it can be scaled horizontally without impacting the other components. This optimizes resource usage and reduces costs, as different types of machines (e.g., GPU versus CPU) can be used according to each service's needs.

For example, let's assume that the LLM inference takes longer, so you will need more ML service replicas to meet the demand. But remember that GPU VMs are expensive. By decoupling the two components, you will run only what is required on the GPU machine and not block the GPU VM with other computing that can be done on a much cheaper machine.

Thus, by decoupling the components, you can scale horizontally as required, with minimal costs, providing a cost-effective solution to your system's needs.

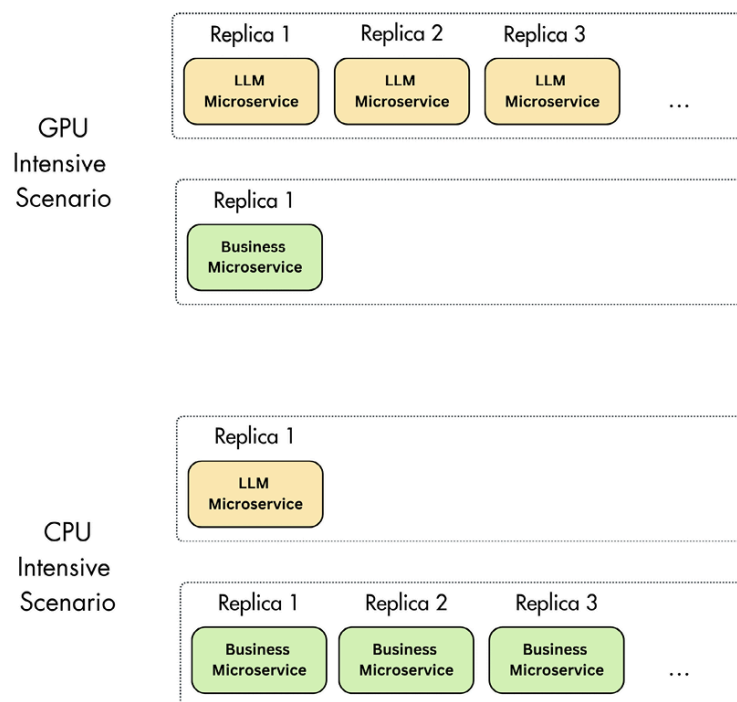


Figure 10.3: Scaling microservices independently based on compute requirements

Additionally, each microservice can adopt the most suitable technology stack, allowing teams to innovate and optimize independently.

However, microservices introduce complexity in deployment and maintenance. Each service must be deployed, monitored, and maintained separately, which can be more challenging than managing a monolithic system.

The increased network communication between services can also introduce latency and potential points of failure, necessitating robust monitoring and resilience mechanisms.

Note that the proposed design for decoupling the ML model and business logic into two services can be extended if necessary. For example, you can have one service for preprocessing the data, one for the model, and an-

other for post-processing the data. Depending on the four pillars (latency, throughput, data, and infrastructure), you can get creative and design the most optimal architecture for your application needs.

Choosing between monolithic and microservices architectures

The choice between monolithic and microservices architectures for serving ML models largely depends on the application's specific needs. A monolithic approach might be ideal for smaller teams or more straightforward applications where ease of development and maintenance is a priority. It's also a good starting point for projects without frequent scaling requirements. Also, if the ML models are smaller, don't require a GPU, or don't require smaller and cheaper GPUs, the trade-off between reducing costs and complicating your infrastructure is worth considering.

On the other hand, microservices, with their adaptability and scalability, are well suited for larger, more complex systems where different components have varying scaling needs or require distinct tech stacks. This architecture is particularly advantageous when scaling specific system parts, such as GPU-intensive LLM services. As LLMs require powerful machines with GPUs, such as Nvidia A100, V100, or A10g, which are incredibly costly, microservices offer the flexibility to optimize the system for keeping these machines busy all the time or quickly scaling down when the GPU is idle. However, this flexibility comes at the cost of increased complexity in both development and operations.

A common strategy is to start with a monolithic design and further decouple it into multiple services as the project grows. However, to successfully do so without making the transition too complex and costly, you must design the monolith application with this in mind. For instance, even if all the code runs on a single machine, you can completely decouple the modules of the application at the software level. This makes it easier to move these modules to different microservices when the time comes. When working with Python, for example, you can implement the ML and business logic into two different Python modules that don't interact with each other. Then, you can glue these two modules at a higher level, such as through a service class, or directly into the framework you use to expose your application over the internet, such as FastAPI.

Another option is to write the ML and business logic as two different Python packages that you glue together in the same ways as before. This is better because it completely enforces a separation between the two but adds extra complexity at development time. The main idea, therefore, is that if you start with a monolith and down the line you want to move to a microservices architecture, it's essential to design your software with modularity in mind. Otherwise, if the logic is mixed, you will probably have to rewrite everything from scratch, adding tons of development time, which translates into wasted resources.

In summary, monolithic architectures offer simplicity and ease of maintenance but at the cost of flexibility and scalability. At the same time, microservices provide the agility to scale and innovate but require more sophisticated management and operational practices.

Exploring the LLM Twin's inference pipeline deployment strategy

Now that we've understood all the design choices available for implementing the deployment strategy of the LLM Twin's inference pipeline, let's explore the concrete decisions we made to actualize it.

Our primary objective is to develop a chatbot that facilitates content creation. To achieve this, we will process requests sequentially, with a strong emphasis on low latency. This necessitates the selection of an online real-time inference deployment architecture.

On the monolith versus microservice aspect, we will split the ML service between a REST API server containing the business logic and an LLM microservice optimized for running the given LLM. As the LLM requires a powerful machine to run the inference, and we can further optimize it with various engines to speed up the latency and memory usage, it makes the most sense to go with the microservice architecture. By doing so, we can quickly adapt the infrastructure based on various LLM sizes. For example, if we run an 8B parameter model, the model can run on a single machine with a Nvidia A10G GPU after quantization. But if we want to run a 30B model, we can upgrade to an Nvidia A100 GPU. Doing so allows us to upgrade only the LLM microservice while keeping the REST API untouched.

As illustrated in *Figure 10.4*, most business logic is centered around RAG in our particular use case. Thus, we will perform RAG's retrieval and augmentation parts within the business microservice. It will also include all the advanced RAG techniques presented in the previous chapter to optimize the pre-retrieval, retrieval, and post-retrieval steps.

The LLM microservice is strictly optimized for the RAG generation component. Ultimately, the business layer will send the prompt trace consisting of the user query, prompt, answer, and other intermediary steps to the prompt monitoring pipeline, which we will detail in *Chapter 11*.

In summary, our approach involves implementing an online real-time ML service using a microservice architecture, which effectively splits the LLM and business logic into two distinct services.

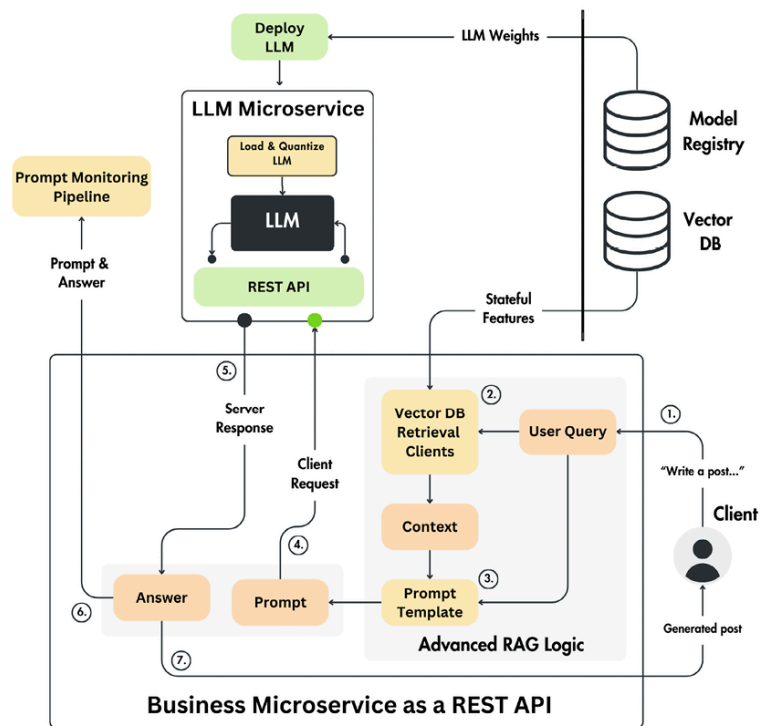


Figure 10.4: Microservice deployment architecture of the LLM Twin's inference pipeline

Let's review the interface of the inference pipeline, which is defined by the **feature/training/inference (FTI)** architecture. For the pipeline to run, it needs two things:

- Real-time features used for RAG, generated by the feature pipeline, which is queried from our online feature store, more concretely from the Qdrant vector database (DB)
- A fine-tuned LLM generated by the training pipeline, which is pulled from our model registry

With that in mind, the flow of the ML service looks as follows, as illustrated in Figure 10.4:

1. A user sends a query through an HTTP request.
2. The user's input retrieves the proper context by leveraging the advanced RAG retrieval module implemented in *Chapter 4*.
3. The user's input and retrieved context are packed into the final prompt using a dedicated prompt template.
4. The prompt is sent to the LLM microservice through an HTTP request.
5. The business microservices wait for the generated answer.
6. After the answer is generated, it is sent to the prompt monitoring pipeline along with the user's input and other vital information to monitor.
7. Ultimately, the generated answer is sent back to the user.

Now, let's explore what tech stack we used to implement the architecture presented in Figure 10.4. As we know, we use Qdrant for the vector DB. We will leverage Hugging Face for the model registry. By doing so, we can publicly share our model with everyone who is testing the code from this book. Thus, you can easily use the model we provided if you don't want to run the training pipeline, which can cost up to 100 dollars. As you can see, shareability and accessibility are some of the most beautiful aspects of storing your model in a model registry.

We will implement the business microservice in FastAPI because it's popular, easy to use, and fast. The LLM microservice will be deployed on AWS SageMaker, where we will leverage SageMaker's integration with Hugging Face's **Deep Learning Containers (DLCs)** to deploy the model. We will discuss Hugging Face's DLCs in the next section, but intuitively, it is an inference engine used to optimize LLMs at serving time. The prompt monitoring pipeline is implemented using Comet, but we will look over that module only in *Chapter 11*.

The SageMaker Inference deployment is composed of the following components that we will show you how to implement:

- **SageMaker endpoint:** An endpoint is a scalable and secure API that SageMaker hosts to enable real-time predictions from deployed models. It's essentially the interface through which applications interact with your model. Once deployed, an application can make HTTP requests to the endpoint to receive real-time predictions.
- **SageMaker model:** In SageMaker, a model is an artifact that results from training an algorithm. It contains the information required to make predictions, including the weights and computation logic. You can create multiple models and use them in different configurations or for various predictions.
- **SageMaker configuration:** This configuration specifies the hardware and software set up to host the model. It defines the resources required for the endpoint, such as the type and number of ML compute instances. Endpoint configurations are used when creating or updating an endpoint. They allow for flexibility in the deployment and scalability of the hosted models.
- **SageMaker Inference component:** This is the last piece of the puzzle that connects the model and configuration to an endpoint. You can deploy multiple models to an endpoint, each with its resource configuration. Once deployed, models are easily accessible via the `InvokeEndpoint` API in Python.

Together, these components create a robust infrastructure for deploying and managing ML models in SageMaker, enabling scalable, secure, and efficient real-time predictions.

Other popular cloud platforms offer the exact solutions. For example, you have Azure OpenAI instead of Bedrock and Azure ML instead of SageMaker on Azure. The list of ML deployment tools, such as Hopsworks, Modal, Vertex AI, Seldon, BentoML, and many more, is endless and will probably change. What is essential though is to understand your use case requirements and find a tool that fits your needs.

The training versus the inference pipeline

Understanding the nuances between the training and inference pipelines is crucial before we deploy the inference pipeline. While it might seem straightforward that the training pipeline is for training and the inference pipeline is for inference, there are significant differences that we need to grasp to comprehend the technical aspects of our discussion fully.

One key difference lies in how data is handled and accessed within each pipeline. During training, data is typically accessed from offline storage in batch mode, optimized for throughput and ensuring data lineage. For ex-

ample, our LLM Twin architecture uses ZenML artifacts to access, version, and track data fed to the training loop in batches. In contrast, the inference pipeline requires an online DB optimized for low latency. We will leverage the Qdrant vector DB to grab the necessary context for RAG. In this context, the focus shifts from data lineage and versioning to quick data access, ensuring a seamless user experience. Additionally, the outputs of these pipelines also differ significantly. The training pipeline outputs trained model weights stored in the model registry. Meanwhile, the inference pipeline outputs predictions served directly to the user.

Also, the infrastructure required for each pipeline is different. The training pipeline demands more powerful machines equipped with as many GPUs as possible. This is because training involves batching data and holding all the necessary gradients in memory for optimization steps, making it highly compute-intensive. More computational power and VRAM allow larger batches (or throughput), reducing training time and enabling more extensive experimentation. On the other hand, the inference pipeline typically requires less computation. Inference often involves passing a single sample or smaller batches to the model without the need for optimization steps.

Despite these differences, there is some overlap between the two pipelines, particularly regarding preprocessing and post-processing steps. Applying the same preprocessing and post-processing functions and hyperparameters during training and inference is crucial. Any discrepancies can lead to what is known as training-serving skew, where the model's performance during inference deviates from its performance during training.

Deploying the LLM Twin service

The last step is implementing the architecture presented in the previous section. More concretely, we will deploy the LLM microservice using AWS SageMaker and the business microservice using FastAPI. Within the business microservice, we will glue the RAG logic written in *Chapter 9* with our fine-tuned LLM Twin, ultimately being able to test out the inference pipeline end to end.

Serving the ML model is one of the most critical steps in any ML application's life cycle, as users can only interact with our model after this phase is completed. If the serving architecture isn't designed correctly or if the infrastructure isn't working properly, it doesn't matter that you have implemented a powerful and excellent model. As long as the user cannot appropriately interact with it, it has near zero value from a business point of view. For example, if you have the best code assistant on the market, but the latency to use it is too high, or the API calls keep crashing, the user will probably switch to a less performant code assistant that works faster and is more stable.

Thus, in this section, we will show you how to:

- Deploy our fine-tuned LLM Twin model to AWS SageMaker
- Write an inference client to interact with the deployed model
- Write the business service in FastAPI
- Integrate our RAG logic with our fine-tuned LLM
- Implement autoscaling rules for the LLM microservice

Implementing the LLM microservice using AWS SageMaker

We aim to deploy the LLM Twin model, stored in Hugging Face's model registry, to Amazon SageMaker as an online real-time inference endpoint. We will leverage Hugging Face's specialized inference container, known as the Hugging Face LLM **DLC**, to deploy our LLM.

What are Hugging Face's DLCs?

DLCs are specialized Docker images that come pre-loaded with essential deep-learning frameworks and libraries, including popular tools like transformers, datasets, and tokenizers from Hugging Face. These containers are designed to simplify the process of training and deploying models by eliminating the need for complex environment setup and optimization. The Hugging Face Inference DLC, in particular, includes a fully integrated serving stack, significantly simplifying the deployment process and reducing the technical expertise needed to serve deep learning models in production.

When it comes to serving models, the DLC is powered by the **Text Generation Inference (TGI)** engine, made by Hugging Face:

<https://github.com/huggingface/text-generation-inference>.

TGI is an open-source solution for deploying and serving LLMs. It offers high-performance text generation using tensor parallelism and dynamic batching for the most popular open-source LLMs available on Hugging Face, such as Mistral, Llama, and Falcon. To sum up, the most powerful features the DLC image provides are:

- **Tensor parallelism**, thus enhancing the computational efficiency of model inference
- **Optimized transformers code for inference**, leveraging flash-attention to maximize performance across the most widely used architectures: <https://github.com/Dao-AILab/flash-attention>
- **Quantization with bitsandbytes** that reduces the model size while maintaining performance, making deployments more efficient: <https://github.com/bitsandbytes-foundation/bitsandbytes>
- **Continuous batching of incoming requests**, thus improving throughput by dynamically batching requests as they arrive
- **Accelerated weight loading** by utilizing **safetensors** for faster model initialization, reducing start-up time: <https://github.com/huggingface/safetensors>
- **Token streaming** that supports real-time interactions through **Server-Sent Events (SSE)**

To summarize, our LLM Twin model will run inside DLC Docker images, listening to requests, optimizing the LLM for inference, and serving the results in real time. The DLC's Docker images will be hosted on AWS SageMaker under inference endpoints that can be accessed through HTTP requests. With that in mind, let's move on to the implementation. We will start by deploying the LLM and then writing a wrapper class to interact with the SageMaker Inference endpoint.

Configuring SageMaker roles

The first step is to create the proper AWS **Identity and Access Management (IAM)** users and roles to access and deploy the SageMaker infrastructure. AWS IAM controls who can authenticate and what any actor has access to. You can create new users (assigned to people) and new roles (assigned to other actors within your infrastructure, such as EC2 VMs) through IAM.

The whole deployment process is automated. We will have to run a few CLI commands, but first, ensure that you have correctly configured the `AWS_ACCESS_KEY`, `AWS_SECRET_KEY`, and `AWS_REGION` environmental variables in the `.env` file. At this step, the easiest way is to use the credentials attached to an admin role as, in the following steps, we will create a set of narrower IAM roles used in the rest of the chapter.

After you configured your `.env` file, we have to:

1. Create an IAM user restricted to creating and deleting only the resources we need for the deployment, such as SageMaker itself, **Elastic Container Registry (ECR)**, and S3. To make it, run the following:

```
poetry poe create-sagemaker-role
```

This command will generate a JSON file called `sagemaker_user_credentials.json` that contains a new AWS access and secret key. From now on, we will use these credentials to deploy everything related to SageMaker to ensure we modify only the resources associated with SageMaker. Otherwise, we could accidentally modify other AWS resources using an admin account, resulting in additional costs or altering other existing projects. Thus, having a narrow role only to your use case is good practice.

The last step is to take the new credentials from the JSON file and update the `AWS_ACCESS_KEY` and `AWS_SECRET_KEY` variables in your `.env` file. You can check out the implementation at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/infrastructure/aws/role_s/create_sagemaker_role.py.

2. Create an IAM execution role. We will attach this role to the SageMaker deployment, empowering it to access other AWS resources on our behalf. This is standard practice for cloud deployments, as instead of authenticating every machine within your credentials, you attach a role that allows them to access only what is necessary from your infrastructure. In our case, we will provide SageMaker access to AWS S3, CloudWatch, and ECR. To create the role, run the following:

```
poetry poe create-sagemaker-execution-role
```

This command will generate a JSON file called `sagemaker_execution_role.json` that contains the **Amazon Resource Name (ARN)** of the newly created role. The ARN is an ID attached to any AWS resource to identify it across your cloud infrastructure. Take the ARN value from the JSON file and update the `AWS_ARN_ROLE` variable from your `.env` file with it. You can check out the implementation at <https://github.com/PacktPublishing/LLM-Engineers->

[Handbook/blob/main/llm_engineering/infrastructure/aws/roles/create_execution_role.py](#).



If you have issues, configure the AWS CLI with the same AWS credentials as in the `.env` file and repeat the process. Official documentation for installing the AWS CLI: <https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2.html>.

By setting the IAM user and role in your `.env` file, we will automatically load them in the settings Python object and use them throughout the following steps. Now, let's move on to the actual deployment.

Deploying the LLM Twin model to AWS SageMaker

The deployment of AWS SageMaker is fully automated through a set of Python classes, which we will cover in this chapter. This section aims to understand how we configure the SageMaker infrastructure directly from Python. Thus, you don't have to run everything step by step, as in a standard tutorial, but only to understand the code.

We can initiate and finalize the entire SageMaker deployment using a simple CLI command: `poe deploy-inference-endpoint`. This command will initialize all the steps presented in *Figure 10.5*, except for creating the SageMaker AWS IAMs we created and configured in the previous step.

In this section, we will walk you through the code presented in *Figure 10.5* that helps us fully automate the deployment process, starting with the `create_endpoint()` function. Ultimately, we will test the CLI command and check the AWS console to see whether the deployment was successful. The SageMaker deployment code is available at https://github.com/PacktPublishing/LLM-Engineers-Handbook/tree/main/llm_engineering/infrastructure/aws/deploy.

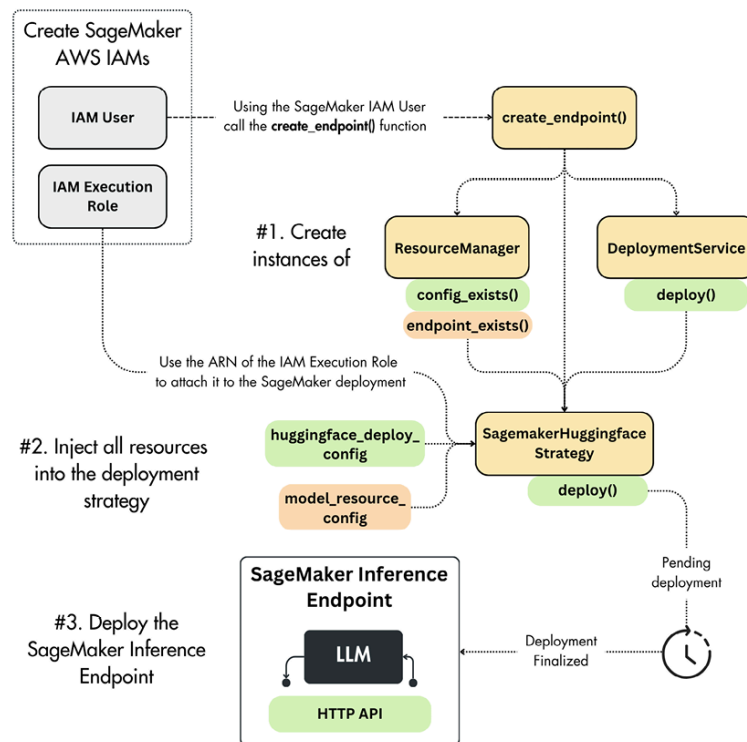


Figure 10.5: AWS SageMaker deployment steps

We will take a top-down approach to walk you through the implementation, starting with the main function that deploys the LLM Twin model to AWS SageMaker. In the function below, we first take the latest version of the Docker DLC image using the `get_huggingface_llm_image_uri()` function, which is later passed to the deployment strategy class, along with an instance of the resource manager and deployment service:

```
def create_endpoint(endpoint_type=EndpointType.INFERENCE_COMPONENT_BASED):
    llm_image = get_huggingface_llm_image_uri("huggingface", version=None)
    resource_manager = ResourceManager()
    deployment_service = DeploymentService(resource_manager=resource_manager)
    SagemakerHuggingfaceStrategy(deployment_service).deploy(
        role_arn=settings.ARN_ROLE,
        llm_image=llm_image,
        config=hugging_face_deploy_config,
        endpoint_name=settings.SAGEMAKER_ENDPOINT_INFERENCE,
        endpoint_config_name=settings.SAGEMAKER_ENDPOINT_CONFIG_INFERENCE,
        gpu_instance_type=settings.GPU_INSTANCE_TYPE,
        resources=model_resource_config,
        endpoint_type=endpoint_type,
    )
```

We must review the three classes used in the `create_endpoint()` function to fully understand the deployment process. Let's start with the `ResourceManager` class. The class begins with the initialization method, establishing the connection to AWS SageMaker using boto3, the AWS SDK for Python, which provides the necessary functions to interact with various AWS services, including SageMaker.

```
class ResourceManager:
    def __init__(self) -> None:
        self.sagemaker_client = boto3.client(
            "sagemaker",
            region_name=settings.AWS_REGION,
```

```

aws_access_key_id=settings.AWS_ACCESS_KEY,
aws_secret_access_key=settings.AWS_SECRET_KEY,
)

```

Next, we implement the `endpoint_config_exists` method, checking whether a specific SageMaker endpoint configuration exists:

```

def endpoint_config_exists(self, endpoint_config_name: str) -> bool:
    try:
        self.sagemaker_client.describe_endpoint_config(EndpointConfigName=endpoint_config_name)
        logger.info(f"Endpoint configuration '{endpoint_config_name}' exists.")
        return True
    except ClientError:
        logger.info(f"Endpoint configuration '{endpoint_config_name}' does not exist.")
        return False

```

The class also includes the `endpoint_exists` method, which checks the existence of a specific SageMaker endpoint:

```

def endpoint_exists(self, endpoint_name: str) -> bool:
    try:
        self.sagemaker_client.describe_endpoint(EndpointName=endpoint_name)
        logger.info(f"Endpoint '{endpoint_name}' exists.")
        return True
    except self.sagemaker_client.exceptions.ResourceNotFoundException:
        logger.info(f"Endpoint '{endpoint_name}' does not exist.")
        return False

```

Let's move to the `DeploymentService`. Within the constructor, we set up the `sagemaker_client`, which will interface with AWS SageMaker and an instance of the `ResourceManager` class we talked about earlier:

```

class DeploymentService:
    def __init__(self, resource_manager):
        self.sagemaker_client = boto3.client(
            "sagemaker",
            region_name=settings.AWS_REGION,
            aws_access_key_id=settings.AWS_ACCESS_KEY,
            aws_secret_access_key=settings.AWS_SECRET_KEY,
        )
        self.resource_manager = resource_manager

```

The `deploy()` method is the heart of the `DeploymentService` class. This method orchestrates the entire process of deploying a model to a SageMaker endpoint. It checks whether the necessary configurations are already in place and, if not, it triggers the deployment:

```

def deploy(
    self,
    role_arn: str,
    llm_image: str,
    config: dict,
    endpoint_name: str,
    endpoint_config_name: str,
    gpu_instance_type: str,
    resources: Optional[dict] = None,
    endpoint_type: enum.Enum = EndpointType.MODEL_BASED,
) -> None:

```

```

try:
    if self.resource_manager.endpoint_config_exists(endpoint_config_name=endpo:
        logger.info(f"Endpoint configuration {endpoint_config_name} exists. Us:
    else:
        logger.info(f"Endpoint configuration{endpoint_config_name} does not ex:
    self.prepare_and_deploy_model(
        role_arn=role_arn,
        llm_image=llm_image,
        config=config,
        endpoint_name=endpoint_name,
        update_endpoint=False,
        resources=resources,
        endpoint_type=endpoint_type,
        gpu_instance_type=gpu_instance_type,
    )
    logger.info(f"Successfully deployed/updated model to endpoint {endpoint_nar
except Exception as e:
    logger.error(f"Failed to deploy model to SageMaker: {e}")
    raise

```

The `deploy` method begins by checking whether the endpoint configuration already exists using the `resource_manager`. This step is crucial because it avoids unnecessary redeployment if the configuration is already set up. The deployment itself is handled by calling the `prepare_and_deploy_model()` method, which is responsible for the actual deployment of the model to the specified SageMaker endpoint.

The `prepare_and_deploy_model()` method is a static method within the `DeploymentService` class. This method is focused on setting up and deploying the Hugging Face model to SageMaker:

```

@staticmethod
def prepare_and_deploy_model(
    role_arn: str,
    llm_image: str,
    config: dict,
    endpoint_name: str,
    update_endpoint: bool,
    gpu_instance_type: str,
    resources: Optional[dict] = None,
    endpoint_type: enum.Enum = EndpointType.MODEL_BASED,
) -> None:
    huggingface_model = HuggingFaceModel(
        role=role_arn,
        image_uri=llm_image,
        env=config,
        transformers_version="4.6",
        pytorch_version="1.13",
        py_version="py310",
    )
    huggingface_model.deploy(
        instance_type=gpu_instance_type,
        initial_instance_count=1,
        endpoint_name=endpoint_name,
        update_endpoint=update_endpoint,
        resources=resources,
        tags=[{"Key": "task", "Value": "model_task"}],
        endpoint_type=endpoint_type,
    )

```

This method begins by creating an instance of `HuggingFaceModel`, a specialized model class from SageMaker designed to handle Hugging Face models. The constructor for `HuggingFaceModel` takes several essential parameters, such as the role ARN (which gives SageMaker the necessary permissions), the URI of the LLM DLC Docker image, and the LLM configuration that specifies what LLM to load from Hugging Face and its inference parameters, such as the maximum total of tokens.

Once `HuggingFaceModel` is instantiated, the method deploys it to SageMaker using the `deploy` function. This deployment process involves specifying the type of instance used, the number of instances, and whether to update an existing endpoint or create a new one. The method also includes optional resources for more complex deployments, such as the `initial_instance_count` parameter for multi-model endpoints and tags for tracking and categorization.

The last step is to walk you through the `SagemakerHuggingfaceStrategy` class, which aggregates everything we have shown. The class is initialized only with an instance of a deployment service, such as the one shown above.

```
class SagemakerHuggingfaceStrategy(DeploymentStrategy):
    def __init__(self, deployment_service):
        self.deployment_service = deployment_service
```

The core functionality of the `SagemakerHuggingfaceStrategy` class is encapsulated in its `deploy()` method. This method orchestrates the deployment process, taking various parameters that define how the Hugging Face model should be deployed to AWS SageMaker:

```
def deploy(
    self,
    role_arn: str,
    llm_image: str,
    config: dict,
    endpoint_name: str,
    endpoint_config_name: str,
    gpu_instance_type: str,
    resources: Optional[dict] = None,
    endpoint_type: enum.Enum = EndpointType.MODEL_BASED,
) -> None:
    logger.info("Starting deployment using Sagemaker Huggingface Strategy...")
    logger.info(
        f"Deployment parameters: nb of replicas: {settings.COPIES}, nb of gpus: {set
```

The parameters passed into the method are crucial to the deployment process:

- `role_arn`: The AWS IAM role that provides permissions for the SageMaker deployment.
- `llm_image`: The URI of the DLC Docker image
- `config`: A dictionary containing configuration settings for the model environment.
- `endpoint_name` and `endpoint_config_name`: Names for the SageMaker endpoint and its configuration, respectively.

- `gpu_instance_type`: The type of the GPU EC2 instances used for the deployment.
- `resources`: Optional resources dictionary used for multi-model endpoint deployments.
- `endpoint_type`: This can either be `MODEL_BASED` or `INFERENCE_COMPONENT`, determining whether the endpoint includes an inference component.

The method delegates the actual deployment process to the `deployment_service`. This delegation is a critical aspect of the strategy pattern, allowing for flexibility in how the deployment is carried out without altering the high-level deployment logic.

```
try:
    self.deployment_service.deploy(
        role_arn=role_arn,
        llm_image=llm_image,
        config=config,
        endpoint_name=endpoint_name,
        endpoint_config_name=endpoint_config_name,
        gpu_instance_type=gpu_instance_type,
        resources=resources,
        endpoint_type=endpoint_type,
    )
    logger.info("Deployment completed successfully.")
except Exception as e:
    logger.error(f"Error during deployment: {e}")
    raise
```

Also, let's review the resource configuration to understand the infrastructure better. These resources are leveraged when setting up multi-endpoint configurations that use multiple replicas to serve clients while respecting the latency and throughput requirements of the application. The `ResourceRequirements` object is initialized with a dictionary that specifies various resource parameters. These parameters include the number of replicas (copies) of the model to be deployed, the number of GPUs required, the number of CPU cores, and the memory allocation in megabytes. Each of these parameters plays a crucial role in the performance and scalability of the deployed model.

```
from sagemaker.compute_resource_requirements.resource_requirements import ResourceRequirements
model_resource_config = ResourceRequirements(
    requests={
        "copies": settings.COPIES,
        "num_accelerators": settings.GPUS,
        "num_cpus": settings.CPUS,
        "memory": 5 * 1024
    },
)
```

In the preceding snippet, `ResourceRequirements` is configured with four key parameters:

- **copies**: This parameter determines how many instances or replicas of the model should be deployed. Having multiple replicas can help in reducing latency and increasing throughput.
- **num_accelerators**: This parameter specifies the number of GPUs to allocate. Since LLMs are computationally intensive, multiple GPUs are

typically required to accelerate inference processes.

- **num_cpus:** This defines the number of CPU cores the deployment should have. The number of CPUs impacts the model's ability to handle data preprocessing, post-processing, and other tasks that are less GPU-dependent but still essential.
- **memory:** The memory parameter sets the minimum amount of RAM required for the deployment. Adequate memory is necessary to ensure the model can load and operate without running into memory shortages.

By setting these parameters, the class ensures that it has sufficient resources to operate efficiently when the model is deployed to a SageMaker endpoint. The precise tuning of these values will vary depending on the LLM's specific requirements, such as its size, the complexity of the tasks it will perform, and the expected load. To get a better understanding of how to use them, after deploying the endpoint, we suggest modifying them and seeing how the performance of the LLM microservice changes.

Ultimately, let's review the settings configuring the LLM engine. The `HF_MODEL_ID` identifies which Hugging Face model to deploy. For example, in the settings class, we set it to `mlabonne/TwinLlama-3.1-8B-13` to load our custom LLM Twin model stored in Hugging Face. `SM_NUM_GPUS` specifies the number of GPUs allocated per model replica, which is crucial for fitting your model into the GPU's VRAM. `HUGGING_FACE_HUB_TOKEN` provides access to the Hugging Face Hub for model retrieval. `HF_MODEL_QUANTIZE` specifies what quantization technique to use, while the rest of the variables control the LLM token generation process.

```
hugging_face_deploy_config = {
    "HF_MODEL_ID": settings.HF_MODEL_ID,
    "SM_NUM_GPUS": json.dumps(settings.SM_NUM_GPUS), # Number of GPU used per rep.
    "MAX_INPUT_LENGTH": json.dumps(settings.MAX_INPUT_LENGTH), # Max length of inp
    "MAX_TOTAL_TOKENS": json.dumps(settings.MAX_TOTAL_TOKENS), # Max length of the
    "MAX_BATCH_TOTAL_TOKENS": json.dumps(settings.MAX_BATCH_TOTAL_TOKENS),
    "HUGGING_FACE_HUB_TOKEN": settings.HUGGINGFACE_ACCESS_TOKEN,
    "MAX_BATCH_PREFILL_TOKENS": "10000",
    "HF_MODEL_QUANTIZE": "bitsandbytes",
}
```

Using these two configurations, we fully control our infrastructure, what LLM to use, and how it behaves. To start the SageMaker deployment with the configuration shown above, call the `create_endpoint()` function (presented at the beginning of the section) as follows:

```
create_endpoint(endpoint_type=EndpointType.MODEL_BASED)
```

For convenience, we also wrapped it up under a `poe` command:

```
poetry poe deploy-inference-endpoint
```

That's all you need to deploy an inference pipeline to AWS SageMaker. The hardest part is finding the correct configuration to fit your needs while reducing your infrastructure's costs. Depending on AWS, this will take up to 15-30 minutes to deploy. You can always change any value di-

rectly from your `.env` file and deploy the model with a different configuration without touching the code. For example, our default values use a single GPU instance of type `ml.g5.xlargeGPU`. If you want more replicas, you can tweak the `GPUS` and `SM_NUM_GPUS` settings or change your instance type by changing the `GPU_INSTANCE_TYPE` variable.



Before deploying the LLM microservice to AWS SageMaker, ensure that you've generated a user role by running `poetry poe create-sagemaker-role` and an execution role by running `poetry poe create-sagemaker-execution-role`. Also, ensure you update your `AWS_*` environment variables in your `.env` file with the credentials generated by the two steps. You can find more details on this aspect in the repository's README file.

After deploying the AWS SageMaker Inference endpoint, you can navigate to the SageMaker dashboard in AWS to visualize it. First, in the left panel, click on **SageMaker dashboard**, and then in the **Inference** column, click on the **Endpoints** button, as illustrated in *Figure 10.6*.

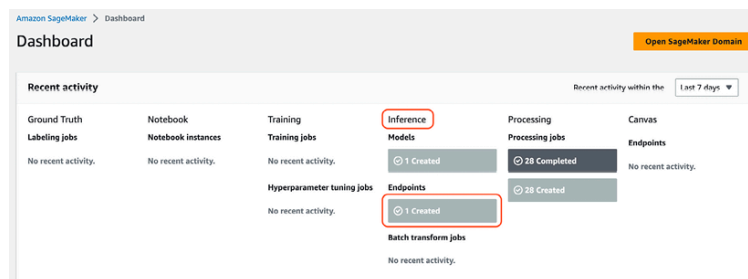


Figure 10.6: AWS SageMaker Inference endpoints example



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

The next-gen Packt Reader and a **free PDF/ePub copy** of this book are included with your purchase. Unlock them by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781836200079>.

After clicking the **Endpoints** button, you will see your **twin** endpoint in a **Creating** or **Created** status, as seen in *Figure 10.7*. After clicking on it, you can look at the endpoint's logs in CloudWatch and monitor the CPU, memory, disk, and GPU utilization. Also, they provide an excellent way to monitor all the HTTP errors, such as `4xx` and `5xx`, in one place.

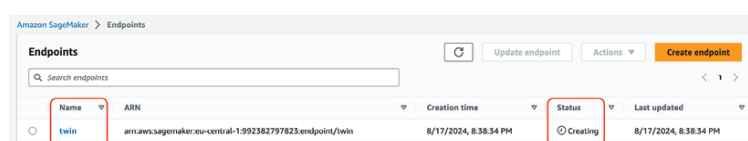


Figure 10.7: AWS SageMaker twin inference endpoint example

Calling the AWS SageMaker Inference endpoint

Now that our LLM service has been deployed on AWS SageMaker, let's learn how to call the service. To do so, we will write two classes that will help us prepare the prompt for SageMaker, call the inference endpoint through HTTP requests, and decode the results in a way the client can work with. All the AWS SageMaker Inference code is available on GitHub at `llm_engineering/model/inference`. It all starts with the following example:

```
text = "Write me a post about AWS SageMaker inference endpoints."
llm = LLMInferenceSagemakerEndpoint(
    endpoint_name=settings.SAGEMAKER_ENDPOINT_INFERENCE
)
Answer = InferenceExecutor(llm, text).execute()
```

As before, we will walk you through the `LLMInferenceSagemakerEndpoint` and `InferenceExecutor` classes. Let's start with the `LLMInferenceSagemakerEndpoint` class, which directly interacts with SageMaker. The constructor initializes all the essential attributes necessary to interact with the SageMaker endpoint:

```
class LLMInferenceSagemakerEndpoint(Inference):
    def __init__(
        self,
        endpoint_name: str,
        default_payload: Optional[Dict[str, Any]] = None,
        inference_component_name: Optional[str] = None,
    ) -> None:
        super().__init__()
        self.client = boto3.client(
            "sagemaker-runtime",
            region_name=settings.AWS_REGION,
            aws_access_key_id=settings.AWS_ACCESS_KEY,
            aws_secret_access_key=settings.AWS_SECRET_KEY,
        )
        self.endpoint_name = endpoint_name
        self.payload = default_payload if default_payload else self._default_payload()
        self.inference_component_name = inference_component_name
```

`endpoint_name` is crucial for identifying the SageMaker endpoint we want to request. Additionally, the method initializes the payload using a provided value or by calling a method that generates a default payload if none is provided.

One of the key features of the class is its ability to generate a default payload for inference requests. This is handled by the

`_default_payload()` method:

```
def _default_payload(self) -> Dict[str, Any]:
    return {
        "inputs": "",
        "parameters": {
            "max_new_tokens": settings.MAX_NEW_TOKENS_INFERENCE,
            "top_p": settings.TOP_P_INFERENCE,
            "temperature": settings.TEMPERATURE_INFERENCE,
            "return_full_text": False,
```

```
    },
}
```

This method returns a dictionary that represents the default structure of the payload to be sent for inference. The parameters section includes settings that influence the model's behavior during inference, such as the number of tokens to generate, the sampling strategy (`top_p`), and the temperature setting, which controls randomness in the output. These parameters are fetched from the application's settings, ensuring consistency across different inference tasks.

The class allows customization of the payload through the `set_payload()` method, which enables the user to modify the inputs and parameters before sending an inference request:

```
def set_payload(self, inputs: str, parameters: Optional[Dict[str, Any]] = None) ->
    self.payload["inputs"] = inputs
    if parameters:
        self.payload["parameters"].update(parameters)
```

This method updates the `inputs` field of the payload with the new input text provided by the user. Additionally, it allows for modifying inference parameters if any are provided.

Ultimately, we leverage the `inference()` method to call the SageMaker endpoint with the customized payload:

```
def inference(self) -> Dict[str, Any]:
    try:
        logger.info("Inference request sent.")
        invoke_args = {
            "EndpointName": self.endpoint_name,
            "ContentType": "application/json",
            "Body": json.dumps(self.payload),
        }
        if self.inference_component_name not in ["None", None]:
            invoke_args["InferenceComponentName"] = self.inference_component_name
        response = self.client.invoke_endpoint(**invoke_args)
        response_body = response["Body"].read().decode("utf8")
        return json.loads(response_body)
    except Exception:
        logger.exception("SageMaker inference failed.")
        raise
```

In this method, the inference method constructs the request to be sent to the SageMaker endpoint. The method packages the payload and other necessary details into a format SageMaker expects. If an `inference_component_name` is specified, it is included in the request, allowing for more granular control over the inference process if needed. The request is sent using the `invoke_endpoint()` function, and the response is read, decoded, and returned as a JSON object.

Let's understand how the `InferenceExecutor` uses the `LLMInferenceSagemakerEndpoint` class we previously presented to send HTTP requests to the AWS SageMaker endpoint.

The `InferenceExecutor` class begins with the constructor, which inputs key parameters for calling the LLM. The `llm` parameter accepts any instance that implements the Inference interface, such as the `LLMInferenceSagemakerEndpoint` class, which is used to perform the inference.

Also, it accepts the query parameter, which represents the user input. Ultimately, it takes an optional context field if you want to do RAG, and you can customize the prompt template. If no prompt template is provided, it will default to a generic version that is not specialized in any LLM:

```
class InferenceExecutor:
    def __init__(
        self,
        llm: Inference,
        query: str,
        context: str | None = None,
        prompt: str | None = None,
    ) -> None:
        self.llm = llm
        self.query = query
        self.context = context if context else ""
        if prompt is None:
            self.prompt = """
            You are a content creator. Write what the user asked you to while using the pro
            User query: {query}
            Context: {context}
            """
        else:
            self.prompt = prompt
```

The `execute()` method is the key component of the `InferenceExecutor` class. This method is responsible for actually performing the inference. When `execute` is called, it prepares the payload sent to the LLM by formatting the prompt with the user's query and context.

Then, it configures several parameters that influence the behavior of the LLM, such as the maximum number of new tokens the model is allowed to generate, a repetition penalty to discourage the model from generating repetitive text, and the temperature setting that controls the randomness of the output.

Once the payload and parameters are set, the method calls the `inference` function from `LLMInferenceSagemakerEndpoint` and waits for the generated answer:

```
def execute(self) -> str:
    self.llm.set_payload(
        inputs=self.prompt.format(query=self.query, context=self.context),
        parameters={
            "max_new_tokens": settings.MAX_NEW_TOKENS_INFERENCE,
            "repetition_penalty": 1.1,
            "temperature": settings.TEMPERATURE_INFERENCE,
        },
    )
    answer = self.llm.inference()[0]["generated_text"]
    return answer
```

By making the inference through an object that implements the Inference interface we decouple, we can easily inject other Inference strategies and the `LLMInferenceSagemakerEndpoint` implementation presented above without modifying different parts of the code.

Running a test example is straightforward. Simply call the following Python file, as shown below:

```
poetry run python -m llm_engineering.model.inference.test
```

Also, for convenience, we wrap it under a `poe` command:

```
poetry poe test-sagemaker-endpoint
```

Now, we must understand how we implement the business microservice using FastAPI. This microservice will send HTTP requests to the LLM microservice defined above and call the RAG retrieval module implemented in *Chapter 9*.

Building the business microservice using FastAPI

To implement a simple FastAPI application that proves our deployment strategy, we first have to define a FastAPI instance as follows:

```
from fastapi import FastAPI
app = FastAPI()
```

Next, we define the `QueryRequest` and `QueryResponse` classes using Pydantic's `BaseModel`. These classes represent the request and response structure for the FastAPI endpoints:

```
class QueryRequest(BaseModel):
    query: str

class QueryResponse(BaseModel):
    answer: str
```

Now that we've defined our FastAPI components and have all the SageMaker elements in place, let's reiterate over the `call_llm_service()` and `rag()` functions we've presented in *Chapter 9* and couldn't run because we haven't deployed our fine-tuned LLM. Thus, as a refresher, the `call_llm_service()` function wraps the inference logic used to call the SageMaker LLM microservice:

```
def call_llm_service(query: str, context: str | None) -> str:
    llm = LLMInferenceSagemakerEndpoint(
        endpoint_name=settings.SAGEMAKER_ENDPOINT_INFERENCE, inference_component_name=...
    )
    answer = InferenceExecutor(llm, query, context).execute()
    return answer
```

Next, we define the `rag()` function that implements all the RAG business logic. To avoid repeating ourselves, check *Chapter 9* for the complete function explanation. What is important to highlight is that the `rag()` function only implements the business steps required to do RAG, which

are CPU- and I/O-bounded. For example, the `ContextRetriever` class makes API calls to OpenAI and Qdrant, which are network I/O bounded, and calls the embedding model, which runs directly on the CPU. Also, as the LLM inference logic is moved to a different microservice, the `call_llm_service()` function is only network I/O bounded. To conclude, the whole function is light to run, where the heavy computing is done on other services, which allows us to host the FastAPI server on a light and cheap machine that doesn't need a GPU to run at low latencies:

```
def rag(query: str) -> str:
    retriever = ContextRetriever(mock=False)
    documents = retriever.search(query, k=3 * 3)
    context = EmbeddedChunk.to_context(documents)
    answer = call_llm_service(query, context)
    return answer
```

Ultimately, we define the `rag_endpoint()` function, used to expose our RAG logic over the internet as an HTTP endpoint. We use a Python decorator to expose it as a POST endpoint in the FastAPI application. This endpoint is mapped to the `/rag` route and expects a `QueryRequest` as input. The function processes the request by calling the `rag` function with the user's query. If successful, it returns the answer wrapped in a `QueryResponse` object. If an exception occurs, it raises an HTTP 500 error with the exception details:

```
@app.post("/rag", response_model=QueryResponse)
async def rag_endpoint(request: QueryRequest):
    try:
        answer = rag(query=request.query)
        return {"answer": answer}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e)) from e
```

This FastAPI application demonstrates how to effectively integrate an LLM hosted on AWS SageMaker into a web service, utilizing RAG to enhance the relevance of the model's responses. The code's modular design, leveraging custom classes like `ContextRetriever`, `InferenceExecutor`, and `LLMInferenceSagemakerEndpoint`, allows for easy customization and scalability, making it a powerful tool for deploying ML models in production environments.

We will leverage the `uvicorn` web server, the go-to method for FastAPI applications, to start the server. To do so, you have to run the following:

```
uvicorn tools.ml_service:app --host 0.0.0.0 --port 8000 --reload
```

Also, you can run the following `poe` command to achieve the same:

```
poetry poe run-inference-ml-service
```

To call the `/rag` endpoint, we can leverage the `curl` CLI command to make a POST HTTP request to our FastAPI server, as follows:

```
curl -X POST 'http://127.0.0.1:8000/rag' -H 'Content-Type: application/json' -d '{}'
```

As usual, we provided an example using a `poe` command that contains an actual user query:

```
poetry poe call-inference-ml-service
```

This FastAPI server runs only locally. The next step would be to deploy it to AWS **Elastic Kubernetes Service (EKS)**, a self-hosted version of Kubernetes by AWS. Another option would be to deploy it to AWS **Elastic Container Service (ECS)**, which is similar to AWS EKS but doesn't use Kubernetes under the hood but AWS's implementation. Unfortunately, this is not specific to LLMs or LLMops. Hence, we won't go through these steps in this book. But to get an idea of what you must do, you must create an AWS EKS/ECS cluster from the dashboard or leverage an **infrastructure-as-code (IaC)** tool such as Terraform. After that, you will have to Dockerize the FastAPI code presented above. Ultimately, you would have to push the Docker image to AWS ECR and create an ECS/EKR deployment using the Docker image hosted on ECR. If this sounds like a lot, the good news is that we will walk you through a similar example in *Chapter 11*, where we will deploy the ZenML pipelines to AWS.



Once you're done testing your inference pipeline deployment, deleting all your AWS SageMaker resources used to deploy the LLM is essential. As almost all AWS resources use a pay-as-you-go strategy, using SageMaker for a few hours wouldn't break your wallet, but if you forget and leave it open, in a few days, the costs can grow exponentially. Thus, a good rule of thumb is to always delete everything after you're done testing your SageMaker infrastructure (or any AWS resource). Luckily, we have provided a script that deletes all the AWS SageMaker resources for you:

```
poetry poe delete-inference-endpoint
```

To ensure everything was correctly deleted, go to your SageMaker dashboard and check it yourself.

Autoscaling capabilities to handle spikes in usage

So far, the SageMaker LLM microservice has used a static number of replicas to serve our users, which means that all the time, regardless of the traffic, it has the same number of instances up and running. As we highlighted throughout this book, machines with GPUs are expensive. Thus, we lose a lot of money during downtime when most replicas are idle. Also, if our application has sudden spikes in traffic, the application will perform poorly as the server cannot handle the number of requests. This is a massive problem for the user experience of our application, as in those spikes, we bring in the majority of new users. Thus, if they have a terrible impression of our product, we significantly reduce their chance of returning to our platform.

Previously, we configured our multi-endpoint service using the `ResourceRequirements` class from SageMaker. For example, let's as-

sume we requested four copies (replicas) with the following compute requirements:

```
model_resource_config = ResourceRequirements(
    requests={
        "copies": 4, # Number of replicas.
        "num_accelerators": 4, # Number of GPUs required.
        "num_cpus": 8, # Number of CPU cores required.
        "memory": 5 * 1024, # Minimum memory required in Mb (required)
    },
)
```

Using this configuration, we always have four replicas serving the clients, regardless of idle time or spikes in traffic. The solution is to implement an autoscaling strategy that scales the number of replicas up and down dynamically based on various metrics, such as the number of requests.

For example, *Figure 10.8* shows a standard architecture where the SageMaker Inference endpoints scale in and out based on the number of requests. When there is no traffic, we can have one online replica so the server remains responsive to new user requests or even scales down to zero if the latency is not super critical. Then, let's assume that when we have around 10 requests per second, we have to keep two replicas online, and when the number of requests spikes to 100 per second, the autoscaling service should spin up to 20 replicas to keep up with the demand. Note that these are fictional numbers that should be adapted to your specific use case.

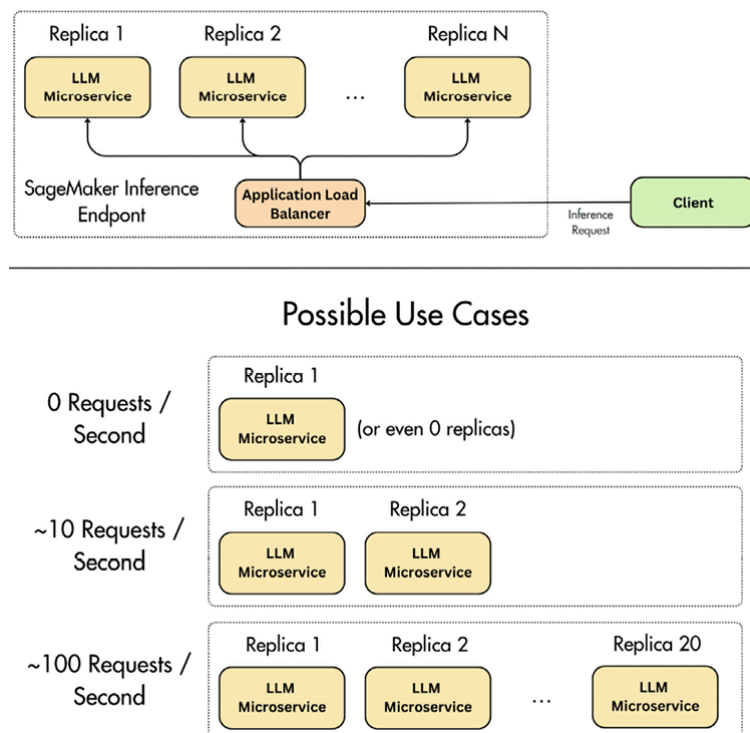


Figure 10.8: Autoscaling possible use cases

Without going into the little details of cloud networking, when working with multi-replica systems, between the client and the replicas sits an **Application Load Balancer (ALB)** or another type of load balancer. All the requests first go to the ALB, which knows to route them to a replica.

The ALB can adopt various routing strategies, where the simplest one is called round robin, which sequentially sends a request to each replica. For example, the first request is routed to replica one, the second to replica two, and so on. Taking this approach, regardless of how many replicas you have online, the endpoint that the client calls is always represented by the load balancer that acts as an entry point into your cluster. Thus, adding or removing new replicas doesn't affect the server and client communication protocol.

Let's quickly learn how to implement an autoscaling strategy for our AWS SageMaker Inference endpoint. SageMaker provides a feature called **Application Auto Scaling** that allows you to scale resources dynamically based on pre-defined policies. Two foundational steps are involved in effectively leveraging this functionality: registering a scalable target and creating a scalable policy.

Registering a scalable target

The first step in enabling autoscaling for your resources is to register a scalable target with the **Application Auto Scaling** feature AWS provides. Think of this as informing AWS about the specific resource you intend to scale, as well as setting the boundaries within which the scaling should occur. However, this step does not dictate how or when the scaling should happen.

For instance, when working with SageMaker Inference components, you'll define the following:

- **Resource ID:** This serves as a unique identifier for the resource you want to scale, typically including the name of the SageMaker Inference component.
- **Service namespace:** This identifies the AWS service the resource belongs to, which, in this case, is **SageMaker**.
- **Scalable dimension:** This specifies the resources to be scaled, such as the desired number of copies.
- **MinCapacity and MaxCapacity:** These parameters define the boundaries of the autoscaling strategies, such as minimum and maximum limits of the number of replicas.

By registering a scalable target, you prepare your SageMaker Inference component for future scaling actions without determining when or how these actions should occur.

Creating a scalable policy

Once your scalable target is registered, the next step is defining how the scaling should occur. This is where creating a scaling policy comes in. A scaling policy defines specific rules that trigger scaling events. When creating policies, you have to define metrics to know what to monitor and thresholds to know when to emit scaling events.

In the context of our SageMaker Inference component, the scalable policy might include the following elements:

- **Policy type:** For instance, you might select **TargetTrackingScaling**, a policy that adjusts the resource's capacity to maintain a specific target value for a chosen metric.

- **Target tracking configuration:** This involves selecting the metric to monitor (such as *SageMakerInferenceComponentInvocationsPerCopy*), setting the desired target value, and specifying cooldown periods that control how quickly scaling actions can occur after previous ones.

The scaling policy defines the rules for your scaling-in and scaling-out strategy. It constantly monitors the specified metric, and depending on whether the metric exceeds or falls below the target value, it triggers actions to scale the number of inference component copies up or down, always within the limits defined by the registered scalable target.

Let's explain in more depth how the **TargetTrackingScaling** policy works. Imagine you have a metric that represents the ideal average utilization or throughput level for your application. With target tracking, you select this metric and set a target value that reflects the optimal state for your application. Once defined, **Application Auto Scaling** creates and manages the necessary CloudWatch alarms to monitor this metric. When deviations occur, scaling actions are triggered, similar to how a thermostat adjusts to maintain a consistent room temperature.

For instance, consider an application running on SageMaker. Let's assume we set a target of keeping GPU utilization around 70 percent. This target allows you to maintain enough headroom to manage sudden traffic spikes while preventing the unnecessary cost of idle resources. When GPU usage exceeds the target, the system scales out, adding resources to manage the increased load. Conversely, when GPU usage drops below the target, the system scales in, reducing capacity to minimize costs during quieter periods.

One significant advantage of setting up target tracking policies using Application Auto Scaling is that they simplify the scaling process. You no longer need to configure CloudWatch alarms and define scaling adjustments manually.

Minimum and maximum scaling limits

When setting up autoscaling for your SageMaker Inference endpoints, it's crucial to establish your maximum and minimum scaling limits before creating your scaling policy. The minimum value represents the least resources your model can operate with. This value must be at least 1, ensuring that your model always has some capacity.

Next, configure the maximum value, which defines the upper limit of resources your model can scale up to. While the maximum must be equal to or greater than the minimum value, it doesn't impose any upper limit. Thus, you can scale up as much as your application needs within the boundaries of what AWS can provide.

Cooldown period

Another important aspect of a scaling policy is the cooldown period, during which it's crucial to maintain a balance between responsiveness and stability. This cooldown period acts as a safeguard, ensuring that your system doesn't overreact during scaling events—whether it's reducing capacity (scaling in) or increasing it (scaling out). By introducing a calculated pause, the cooldown period prevents rapid fluctuations in the number of instances. Specifically, it delays the removal of instances during scale-

in requests and restricts the creation of new replicas during scale-out requests. This strategy helps maintain a stable and efficient environment for LLM service.

These practical basics are used in autoscaling most web servers, including online real-time ML servers. Once you understand how to configure scaling policies for SageMaker, you can immediately apply the strategies you've learned to other popular deployment tools like Kubernetes or AWS ECS.



For a step-by-step guideline on how to configure autoscaling for the AWS SageMaker endpoint implemented in this chapter, you can follow this official tutorial from AWS:

<https://docs.aws.amazon.com/sagemaker/latest/dg/endpoint-auto-scaling-prerequisites.html>.

Autoscaling is a critical component in any cloud architecture, but there are some pitfalls you should be aware of. The first and most dangerous one is over-scaling, which directly impacts the costs of your infrastructure. If your scaling policy or cooldown period is too sensitive, you may be uselessly spinning up new machines that will remain idle or with the resources underused. The second reason is on the other side of the spectrum, where your system doesn't scale enough, resulting in a bad user experience for the end user.

That's why a good practice is to understand the requirements of your system. Based on them, you should tweak and experiment with the autoscaling parameters in a dev or test environment until you find the sweet spot (similar to hyperparameter tuning when training models). Let's suppose, for instance, that you expect your system to support an average of 100 users per minute and scale up to 10,000 users per minute in case of an outlier event such as a holiday. Using this spec, you can stress test your system and monitor your resources to find the best trade-off between costs, latency, and throughput that supports standard and outlier use cases.

Summary

In this chapter, we learned what design decisions to make before serving an ML model, whether an LLM or not, by walking you through the three fundamental deployment types for ML models: online real-time inference, asynchronous inference, and offline batch transform. Then, we considered whether building our ML-serving service as a monolith application made sense or splitting it into two microservices, such as an LLM microservice and a business microservice. To do this, we weighed the pros and cons of a monolithic versus microservices architecture in model-serving.

Next, we walked you through deploying our fine-tuned LLM Twin to an AWS SageMaker Inference endpoint. We also saw how to implement the business microservice using FastAPI, which consists of all the RAG steps based on the retrieval module implemented in *Chapter 9* and the LLM microservice deployed on AWS SageMaker. Ultimately, we explored why we have to implement an autoscaling strategy. We also reviewed a popular autoscaling strategy that scales in and out based on a given set of metrics and saw how to implement it in AWS SageMaker.

In the next chapter, we will learn about the fundamentals of MLOps and LLMOps and then explore how to deploy the ZenML pipelines to AWS and implement a **continuous training**, **continuous integration**, and **continuous delivery (CT/CI/CD)** and monitoring pipeline.

References

- AWS Developers. (2023, September 22). *Machine Learning in 15: Amazon SageMaker High-Performance Inference at Low Cost* [Video]. YouTube. <https://www.youtube.com/watch?v=FRbcb7CtIOw>
- bitsandbytes-foundation. (n.d.). GitHub—bitsandbytes-foundation/bitsandbytes: Accessible large language models via k-bit quantization for PyTorch. GitHub. <https://github.com/bitsandbytes-foundation/bitsandbytes>
- *Difference between IAM role and IAM user in AWS*. (n.d.). Stack Overflow. <https://stackoverflow.com/questions/46199680/difference-between-iam-role-and-iam-user-in-aws>
- Huggingface. (n.d.-a). GitHub—huggingface/safetensors: Simple, safe way to store and distribute tensors. GitHub. <https://github.com/huggingface/safetensors>
- Huggingface. (n.d.-b). GitHub—huggingface/text-generation-inference: Large Language Model Text Generation Inference. GitHub. <https://github.com/huggingface/text-generation-inference>
- Huyen, C. (n.d.). *Designing machine learning systems*. O'Reilly Online Learning. <https://www.oreilly.com/library/view/designing-machine-learning/9781098107956/>
- Iusztin, P. (2024, August 20). Architect LLM & RAG inference pipelines | Decoding ML. *Medium*. <https://medium.com/decodingml/architect-scalable-and-cost-effective-llm-rag-inference-pipelines-73b94ef82a99>
- Lakshmanan, V., Robinson, S., and Munn, M. (n.d.). *Machine Learning design patterns*. O'Reilly Online Learning. <https://www.oreilly.com/library/view/machine-learning-design/9781098115777/>
- Mendoza, A. (2024, August 21). *Best tools for ML model Serving*. neptune.ai. <https://neptune.ai/blog/ml-model-serving-best-tools>

Join our book's Discord space

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/llmeng>

